PROJECT: PERFORMING A CODE REVIEW

datalab

For the past five years, you've honed your skills as a Senior Data Scientist for a global university. Your team leverages its data analytics and machine learning skill sets to help other departments make data-driven decisions. One such department is the procurement team, who is trying to decide the best new mobile phone to offer to the university's employees. For the last week, a Junior Data Scientist on your team has been developing a workflow to help provide insight to the procurement team. You will be reviewing their code to ensure it's ready to ship to production.

The first chunk of code that you'll be reviewing is your colleague's function to prepare smartphone data from a CSV file for visualization. After ingesting and cleaning the smartphone data, your colleague has prepared a function to plot a variable passed to the function, versus `"price"`. However, within this function, there is code that does not adhere to DRY principles and is copied and pasted. Make sure to refactor the code appropriately, using the `column_to_label()` function defined below.

Wow, your colleague even included a unit test to ensure `NaN` values were removed from the cleaned DataFrame! However, it doesn't seem like the unit test is passing when executed. Re-work this unit test to ensure that it matches the transformation logic in the `prepare_smartphone_data()` function.

Once you've made changes to the `test_nan_values` unit test, you'll want to ensure that these unit tests execute with `ExitCode.OK`. This means that the `pytest` defined above has passed testing, and the code is one step closer to being to be shipped to production.

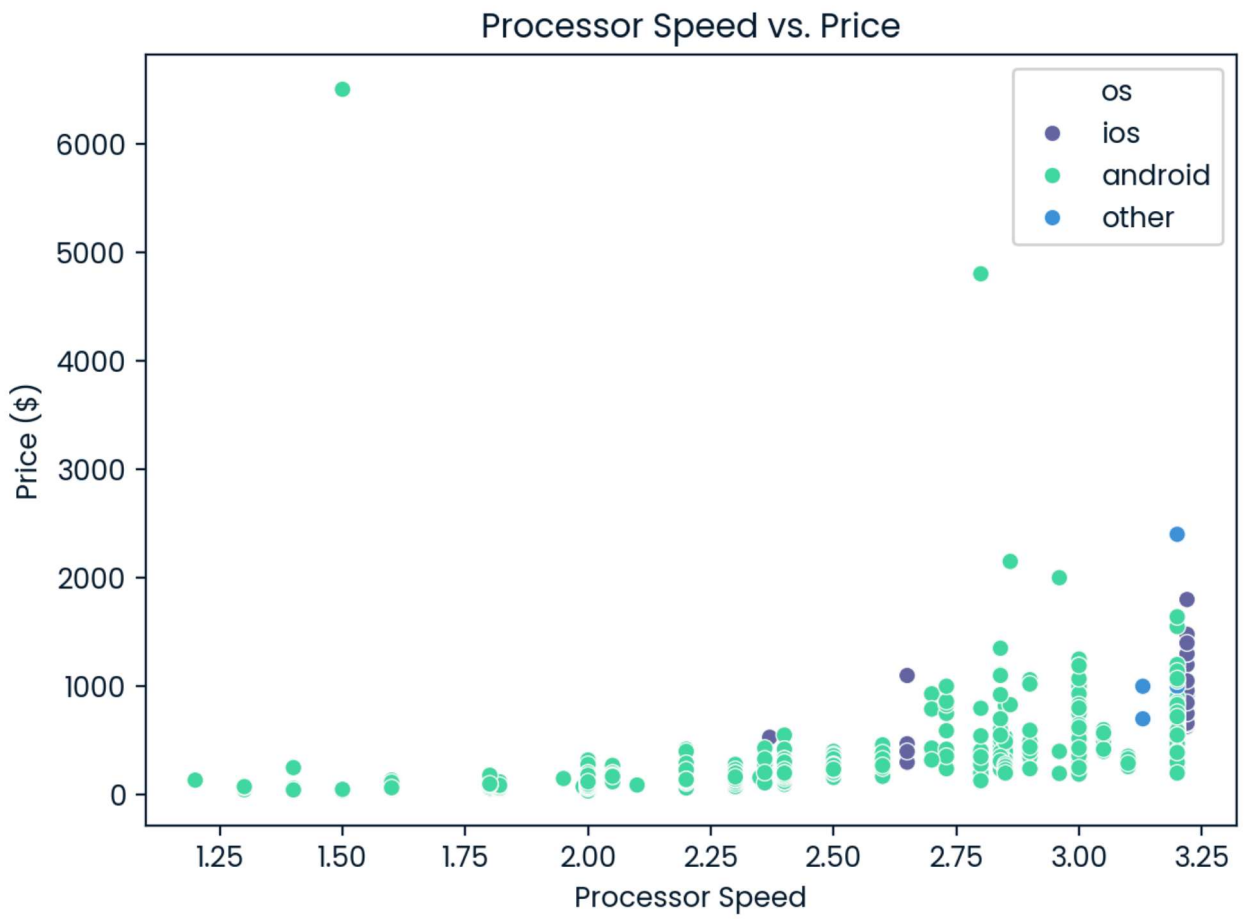
For context, there is a print statement in the `prepare_smartphone_data()` function in the first cell of the notebook below that can be used to visualize the dataset your Junior Data Engineer has been working with. Feel free to update this line of code as needed. This can then be removed after the dataset has been investigated. Best of luck!

```python
import os
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import pytest

def column_to_label(column_name):
    """
    Converts a column name in a pandas DataFrame to a string that can be
    used as a label in a plot.

    :param column_name: string containing original column name
    :return: string that is ready to be presented on a plot
    """
    if isinstance(column_name, str):
        return " ".join(column_name.split("_")).title()
    else:
        raise ValueError("Column name must be a string.")

def prepare_smartphone_data(file_path):
    """
    Prepares the smartphone data for visualization by applying necessary
    transformations after reading the raw DataFrame into memory:
        - reducing the number of columns to only those needed for analysis
        - removing records without a battery_capacity or os value
        - dividing the price column by 100 to find the dollar amount

    :param file_path: the file path where the raw smartphone data is stored
    :return: a cleaned pandas DataFrame
    """
    if not os.path.exists(file_path):
        raise FileNotFoundError(f"File not found at {file_path}")

    raw_data = pd.read_csv(file_path)

    feature_set = [
        "brand_name",
        "os",
        "price",
        "avg_rating",
        "processor_speed",
        "battery_capacity",
        "screen_size"
    ]
    clean_data = raw_data.loc[:, feature_set]

    # Remove entries without 'battery_capacity' or 'os'
```

```
    clean_data = clean_data.dropna(subset=["battery_capacity", "os"])

    # Convert 'price' column to reflect dollar amounts
    clean_data["price"] = clean_data["price"] / 100

    return clean_data

def visualize_versus_price(clean_data, x):
    """
    Use seaborn and matplotlib to identify a pattern between price and
    a specified variable.

    :param clean_data: a pandas DataFrame containing cleaned smartphone data
    :param x: variable to be plotted on the x-axis
    :return: None
    """
    # DRY Refactor: Use column_to_label for consistent axis/title formatting
    x_title = column_to_label(x)
    y_title = column_to_label("price")

    sns.scatterplot(x=x, y="price", data=clean_data, hue="os")
    plt.xlabel(x_title)
    plt.ylabel(f"{y_title} ($)")
    plt.title(f"{x_title} vs. {y_title}")
    plt.tight_layout()
    plt.show()

@pytest.fixture()
def clean_smartphone_data():
    """Fixture to provide cleaned data for testing."""
    return prepare_smartphone_data("./data/smartphones.csv")

def test_nan_values(clean_smartphone_data):
    """
    Ensures no NaN values remain in the columns explicitly targeted by
    the dropna transformation logic in prepare_smartphone_data().
    """
    # The prepare function drops rows where these specific columns are NaN.
    # We assert that after transformation, both columns contain zero nulls.
    assert clean_smartphone_data["battery_capacity"].isna().sum() == 0, \
        "NaN values found in battery_capacity after cleaning."
    assert clean_smartphone_data["os"].isna().sum() == 0, \
        "NaN values found in os after cleaning."

if __name__ == "__main__":
    # 1. Quick dataset inspection (can be removed after validation)
    print("📊 Loading and inspecting cleaned dataset...")
    cleaned_df = prepare_smartphone_data("./data/smartphones.csv")
    print(cleaned_df.head())
```

```python
    print(f"\n✅ Shape after cleaning: {cleaned_df.shape}")

    # 2. Visualization
    print("\n📈 Generating scatterplot...")
    visualize_versus_price(cleaned_df, "processor_speed")

    # 3. Run unit tests
    print("\n🧪 Running unit tests...")
    # Fixed: Removed hardcoded '__file__ = "notebook.ipynb"' and ipytest config.
    # pytest now correctly discovers tests in this .py file.
    exit_code = pytest.main([__file__, "-v", "--tb=short"])

    if exit_code == 0:
        print("\n🎉 All tests passed! Code is ready for production.")
    else:
        print("\n⚠️ Tests failed. Review assertions above.")
```

```
📊 Loading and inspecting cleaned dataset...
   brand_name   os    price  ...  processor_speed  battery_capacity  screen_size
0       apple  ios   389.99  ...             2.65            3110.0          6.1
1       apple  ios   469.99  ...             2.65            3110.0          6.1
2       apple  ios  1099.00  ...             2.65            3500.0          6.5
11      apple  ios   629.99  ...             3.22            3240.0          6.1
12      apple  ios   729.99  ...             3.22            3240.0          6.1

[5 rows x 7 columns]

✅ Shape after cleaning: (955, 7)

📈 Generating scatterplot...
```




```
🧪 Running unit tests...
============================= test session starts ==============================
platform linux -- Python 3.8.10, pytest-7.1.2, pluggy-1.0.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /work/files/workspace
plugins: dash-2.13.0, web3-5.31.0, anyio-3.6.1
collecting ... collected 0 items

============================ no tests ran in 0.06s =============================

⚠️  Tests failed. Review assertions above.
```

```
ERROR: not found: /work/files/workspace/notebook.ipynb
(no name '/work/files/workspace/notebook.ipynb' in any of [])
```